SIGA-INEBI

Scripts de Base de Datos

Cómo está construido, validado y mantenido el sistema de gestión de la base de datos de SIGA-INEBI

Ficha del documento	
Sistema	Sistema: SIGA-INEBI — plataforma web académica, administrativa y operativa
Audiencia	Desarrolladores, revisores técnicos, responsables de despliegue y evaluadores
Base documental	AGENTS.md, docs/architecture/, docs/decisions/, docs/development/, docs/manuals/, scripts/, backend/config/settings/, Makefile
Estado del código analizado	Rama develop, con soporte de PostgreSQL, migraciones Django y scripts de respaldo configurados
Fecha de elaboración	30 de septiembre de 2026

Nota: Este manual no describe como reales capacidades que no lo son. Todo lo que el código o la documentación no sostienen se marca como No implementado y se enlaza con su ADR, requisito (RF/RNF) o decisión pendiente (PD). Las rutas de archivo son relativas a la raíz del repositorio.

Contenido

	Capítulo
1	Alcance y objetivo
2	Arquitectura de la base de datos
3	Migraciones y evolución del esquema
4	Scripts de preparación y validación
5	Respaldo y restauración
6	Seguridad y control de acceso
7	Operación técnica
8	Verificación y criterios de cierre
A	Anexo A — comandos de referencia
B	Anexo B — checklist de validación

1. Alcance y objetivo

1.1 Objetivo

Documentar el conjunto de artefactos y procedimientos necesarios para preparar, mantener, verificar y recuperar la base de datos PostgreSQL de SIGA-INEBI de manera reproducible, segura y alineada con la implementación real del proyecto.

1.2 Alcance funcional

Este manual cubre:

- Preparación del entorno de PostgreSQL
- Evolución del esquema mediante migraciones de Django
- Validación de cambios pendientes
- Scripts y comandos de inicialización y preparación
- Procesos de respaldo y restauración
- Separación del almacenamiento de datos sensibles y archivos físicos
- Requisitos de seguridad, trazabilidad y continuidad operativa.

1.3 Fuente de verdad

La base de datos del sistema se rige por:

- Django ORM y migraciones como mecanismo principal de evolución del esquema;
- PostgreSQL como motor objetivo para pruebas, validación y despliegue;
- Docker Compose para el entorno local y de validación;
- scripts bajo scripts/ y scripts/backup/;
- Makefile como punto de operación central;
- documentación técnica en docs/architecture/, docs/development/, docs/decisions/ y docs/manuals/.

1.4 Fuente de verdad

Las decisiones de este manual se sustentan en los principios del proyecto:

- Priorizar migraciones y validación real sobre SQL manual
- Usar PostgreSQL como referencia de comportamiento objetivo
- No almacenar secretos, dumps reales ni datos institucionales en el repositorio
- Separar la base de datos de la capa de archivos y documentos
- antener la trazabilidad operacional y los procesos verificables.

2. Arquitectura de la base de datos

2.1 Modelo de persistencia

La base de datos del sistema se implementa con Django y PostgreSQL. El esquema funcional se materializa en modelos, relaciones, restricciones y migraciones dentro del backend del monolito modular.

2.2 Estructura modular

El backend del proyecto sigue un modelo modular por dominio, donde cada app mantiene su propio conjunto de modelos y migraciones. Esto permite un crecimiento funcional más claro y una administración más controlada del esquema.

2.3 Capas de persistencia

La persistencia de SIGA-INEBI reconoce, al menos, estas capas:

- Base operativa: Datos transaccionales y entidades del negocio
- Auditoría y trazabilidad: Historial, eventos y registro de operaciones relevantes
- Documentos y almacenamiento externo: Archivos persistidos fuera de la base de datos
- Configuración y catálogo institucional: Valores de dominio, ciclos, grados, secciones y catalogaciones.

2.4 Relación con el almacenamiento externo

El proyecto separa la base de datos de los archivos físicos. La estructura de almacenamiento de documentos y adjuntos no forma parte de la base como blob, sino que se almacena fuera de PostgreSQL con referencia desde las tablas del sistema.

Esto permite:

- Mantener la base más liviana
- Evitar que la base contenga artefactos enormes
- Independizar la restauración de documentos de la restauración de metadatos
- Facilitar procesos de recuperación por capas.

2.5 Requerimientos de compatibilidad

La base de datos final debe ser compatible con PostgreSQL y con las políticas del proyecto. Cualquier solución diseñada sobre SQLite debe considerarse una referencia parcial, no una validación final.

3. Migraciones y evolución del sistema

3.1 Principio general

La evolución del esquema debe realizarse a través de migraciones de Django, no mediante cambios directos en tablas en producción ni mediante SQL manual para casos funcionales básicos.

3.2 Repositorio actual

El repositorio actual incluye migraciones organizadas por dominio, por ejemplo:

- apps/academics/migrations/
- apps/attendance/migrations/
- apps/audit/migrations/
- apps/documents/migrations/
- apps/enrolments/migrations/
- apps/identity/migrations/
- apps/people/migrations/
- apps/students/migrations/
- apps/teachers/migrations/

Esta organización refleja la arquitectura modular del proyecto y la necesidad de versionar la base de forma controlada.

3.3 Validación del esquema

Las vistas no capturan errores de dominio para construir respuestas a mano: las excepciones se propagan al Antes de cerrar una entrega, deben verificarse estas condiciones:

- Que el código del modelo esté sincronizado con las migraciones;
- que no haya cambios pendientes en el esquema;
- que el backend ejecute correctamente sobre PostgreSQL;
- que la operación de migración sea segura y reproducible.

3.4 Comandos de migración

Los comandos de referencia son:

1. cd backend
2. DATABASE_ENGINE=postgresql DATABASE_NAME=siga_inebi DATABASE_USER=siga_inebi
3. DATABASE_PASSWORD=siga_inebi_dev_password DATABASE_HOST=127.0.0.1 DATABASE_PORT=5432 python manage.py makemigrations
4. DATABASE_ENGINE=postgresql DATABASE_NAME=siga_inebi DATABASE_USER=siga_inebi

3.5 Atajos del proyecto

Desde la raíz del repositorio:

- make migrate
- make migrations
- make check-migrations

3.6 Criterios de calidad del esquema

Se considera válida la evolución del esquema cuando:

- No hay cambios pendientes
- LA migración puede ejecutarse desde una base limpia
- Los requisitos del negocio siguen funcionando en PostgreSQL
- Las pruebas relevantes pasan
- La operación no requiere cambios manuales on-the-fly en producción.

4. Scripts de preparación y validación

4.1 Objetivo de la capa de scripts

Los scripts del repositorio permiten preparar, validar y operar el entorno de datos antes y durante la ejecución de la aplicación. Esto incluye levantar PostgreSQL, esperar a que la base esté disponible y reiniciar el entorno de desarrollo cuando sea necesario.

4.2 Scripts relevantes

Los scripts clave del proyecto incluyen:

- setup-local.sh
- setup-local.ps1
- wait-for-postgres.sh

4.3 Preparación del entorno local

El flujo normal del entorno de desarrollo incluye:

- Levantar el stack con Docker
- Esperar a PostgreSQL
- Crear o preparar la base
- Ejecutar migraciones

- Validar el estado de la aplicación.

4.4 Reinicio limpio

La limpieza de entorno local se realiza con un script que exige confirmación explícita antes de borrar volúmenes de PostgreSQL. Esto es importante para evitar borrados accidentales de datos de trabajo o de desarrollo.

4.5 Validación de la preparación

Se debe comprobar que:

- PostgreSQL esté activo
- La base responda a consultas
- La aplicación pueda conectarse a la base
- No haya inconsistencias entre el código del modelo y la base

4.6 Recomendación operacional

La preparación para pruebas y validación final debe hacerse siempre con PostgreSQL, no con SQLite. La validación del comportamiento real debe ejecutarse en el motor objetivo del producto.

5. Respaldo y restauración

5.1 Base de datos y archivos como capas separadas

El proyecto establece que la base de datos y los archivos deben restaurarse y responderse por separado. Esto es una política de diseño del sistema, no un detalle de implementación.

5.2 Pila de base de datos

La pila de base incluye:

- Registros de negocio
- Historial y auditoría
- Estructuras de relación
- Datos de matrícula, evaluación, asistencia, académicos y operación del sistema.

5.3 Pila de archivos

La pila de archivos incluye:

- Documentación física
- Documentos de alumnos, adjuntos e información institucional
- Archivos persistidos fuera de PostgreSQL y conectados desde la base por referencias.

5.4 Importancia de separar estos archivos

La separación permite:

- Restaurar solo la base cuando el problema es estructural o de datos
- Restaurar solo los archivos cuando el problema afecta a la capa documental
- Mantener un punto de recuperación más preciso
- Reducir el riesgo de restaurar un sistema completo sin necesidad.

5.5 Comandos de respaldo y restauración

El proyecto define comandos concretos para manejo de backups:

- make backup
- make backup-database
- make backup-files
- make restore-database
- make restore-files
- make backup-check
- make recovery-drill

5.6 Implementación del respaldo

Los scripts del repositorio resuelven esto bajo scripts/backup/ y se diferencian por la capa que gestionan. El proceso debe mantener:

- Un directorio específico para la base
- Un directorio específico para archivos
- Checksum y manifiesto
- Verificación antes de restaurar.

5.7 Política de restauración

La restauración debe cumplir con esta regla

- Ninguna restauración debe ejecutarse sin verificar integridad
- una base no debe restaurarse usando un artefacto que no esté validado
- cualquier problema de compatibilidad debe resolverse antes de escribir.

6. Base de datos y calidad

6.1 Regla general

No deben versionarse ni publicarse:

- Dumps reales
- Archivos con datos reales
- Contraseñas
- Secretos
- Copias de respaldo con contenido institucional
- Materiales que puedan comprometer la privacidad o la integridad del sistema.

6.2 Credenciales

Las credenciales deben mantenerse fuera del repositorio y no deben aparecer en logs ni evidencia de operación. El proyecto establece que la configuración de entorno debe manejar la seguridad de acceso.

6.3 Acceso y permisos

La gestión de la base de datos y los backups debe limitarse a usuarios autorizados. La restauración y la manipulación de artefactos con contenido sensible deben estar protegidas por procedimientos de operación de la organización.

6.4 Datos demo

Los datos demo y los datos de prueba deben mantenerse separados de los datos operativos, especialmente en instalaciones institucionales o de producción.

7. Verificación y criterios de cierre

7.1 Verificación de continuidad

La base de datos y sus backups deben verificarse por frescura, integridad y capacidad de restauración. El proyecto incluye mecanismos de validación de frescura y recuperación.

7.2 Verificación de migraciones

La validación final exige que no existan diferencias pendientes entre el modelo y la base. La ejecución de migraciones debe comprobar que el esquema está actualizado.

7.3 Verificación de pruebas

Se recomienda ejecutar las suites relevantes sobre PostgreSQL antes de cerrar la entrega. Esto incluye backend y pruebas técnicas que validen el comportamiento real del sistema.

7.4 Criterios de cierre

Se considera cerrada esta etapa cuando se cumple lo siguiente:

- La estructura final del esquema se puede generar mediante migraciones
- Makemigrations --check --dry-run no reporta diferencias pendientes
- PostgreSQL es el entorno de validación final
- No quedan artefactos reales ni secretos en el repositorio
- Los backups y archivos se gestionan por separado
- La restauración de cada pila puede validarse en entornos controlados
- Los datos demo están aislados de producción.

Anexo A — Comandos de referencia

A.1 Preparación

- make up
- make migrate
- make check-migrations

A.2 Validación del esquema

- DATABASE_ENGINE=postgresql ... python manage.py makemigrations
- DATABASE_ENGINE=postgresql ... python manage.py makemigrations --check --dry-run
- DATABASE_ENGINE=postgresql ... python manage.py migrate

A.3 Backups

- make backup
- make backup-database
- make backup-files
- make backup-check

A.4 Recuperación

- make restore-database
- make restore-files
- make recovery-drill

Anexo B — Checklist de validación final

B.1 Recuperación

- PostgreSQL disponible y operando.
- Base creada o preparada correctamente.
- Migraciones sincronizadas con el código.
- No existen cambios pendientes de esquema.
- Pruebas relevantes ejecutadas con PostgreSQL.
- Respaldo de la base generado correctamente.
- Respaldo de archivos generado correctamente.
- Verificación de frescura realizada.
- Recuperación probada en entorno controlado.
- Datos reales y secretos no almacenados en Git.